OPERATING SYSTEMS
ORANGE PROBLEM

Name : Karan Varshney
SRN : PES2UG24CS903
Sec : C
Sem : IV

GitHub Repository:
https://github.com/karan21v/PES2UG24CS903-pes-vcs

PART-A — Screenshots

Phase 1 — Object Storage Foundation

Phase 1 focuses on building the content-addressable object store — the heart of PES-VCS. Every file, directory snapshot, and commit is stored as an object identified by its SHA-256 hash. The two functions implemented are object_write (store data atomically) and object_read (retrieve and verify data).

Screenshot 1A — Object Store Tests

Q: Implement object_write and object_read. Show that blob storage, deduplication, and integrity checking all work correctly.

```
karan@karanvm:~/PES2UG24CS903-pes-vcs$ make all
rm -rf .pes
./test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
gcc -o test_objects test_objects.o object.o -lcrypto
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
karan@karanvm:~/PES2UG24CS903-pes-vcs$
```

All three Phase 1 tests pass. Blob storage confirms that data written to the store is read back identically. Deduplication confirms that writing the same content twice stores only one object — the second write detects the hash already exists and skips it. Integrity check confirms that if an object file is corrupted on disk, recomputing its SHA-256 produces a different hash than the filename, so the corruption is detected and the read fails safely. The object is stored at .pes/objects/d5/8213f5... showing the two-character shard prefix.

Screenshot 1B — Sharded Directory Structure

Q: Show the .pes/objects directory structure after initialising the repository and writing objects.

```
karan@karanvm:~/PES2UG24CS903-pes-vcs$ ./pes init
find .pes/objects -type f
Initialized empty PES repository in .pes/
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
karan@karanvm:~/PES2UG24CS903-pes-vcs$
```

After ./pes init and running the object tests, three objects appear at paths like .pes/objects/2a/594d..., .pes/objects/25/ef1f..., and .pes/objects/d5/8213.... The first two hex characters of each hash form the shard subdirectory. This sharding avoids filesystem limits on files per directory — without it, millions of objects would all sit in one flat directory, making stat() and readdir() very slow. Each object file stores a header (type + size + null byte) followed by the raw content, and the filename is the SHA-256 of the entire file.

Phase 2 — Tree Objects

Phase 2 builds tree objects — the binary data structure that represents a directory snapshot. Each tree entry contains a file mode, a name, and a 32-byte raw SHA-256 hash pointing to either a blob (file) or another tree (subdirectory). The key function implemented is tree_from_index, which recursively converts the flat index into a nested tree hierarchy.

Screenshot 2A — Tree Serialization Tests

Q: Implement tree_parse, tree_serialize, and tree_from_index. Show that tree serialization produces correct and deterministic output.

```
karan@karanvm:~/PES2UG24CS903-pes-vcs$ make all
./test_tree
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
In file included from /usr/include/string.h:535,
                 from tree.c:5:
In function ‘strncpy’,
    inlined from ‘write_tree_level’ at tree.c:109:13:
/usr/include/aarch64-linux-gnu/bits/string_fortified.h:95:10: warning: ‘__builtin_strncpy’ output may be truncated copying 255 bytes from a string of leng
th 255 [-Wstringop-truncation]
   95 |   return __builtin___strncpy_chk (__dest, __src, __len,
      |          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   96 |                                   __glibc_objsize (__dest));
      |                                   ~~~~~~~~~~~~~~~~~~~~~~~~~
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
karan@karanvm:~/PES2UG24CS903-pes-vcs$
```

Both Phase 2 tests pass. The serialize/parse roundtrip test confirms that a Tree struct converted to binary and parsed back produces identical entries, modes, and hashes. The deterministic serialization test confirms that entries are always sorted alphabetically by name before serialization — this is critical because two trees with identical entries in different insertion order must hash to the same value. Without sorting, adding files in different orders would create different tree objects for the same logical directory, breaking deduplication.

Screenshot 2B — Raw Binary Object (xxd)

Q: Inspect the raw binary format of an object stored on disk using xxd.

```
karan@karanvm:~/PES2UG24CS903-pes-vcs$ ./pes init
echo "hello world" > file1.txt
./pes add file1.txt
find .pes/objects -type f
Initialized empty PES repository in .pes/
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
karan@karanvm:~/PES2UG24CS903-pes-vcs$ xxd .pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d | head -20
00000000: 626c 6f62 2031 3200 6865 6c6c 6f20 776f  blob 12.hello wo
00000010: 726c 640a                                rld.
karan@karanvm:~/PES2UG24CS903-pes-vcs$
```

The hex dump shows the blob object wire format: the ASCII text 'blob 12' (type + space + size), followed by a null byte 0x00 as a separator, followed by the raw file content 'hello world\n'. This is identical to Git's blob format. The SHA-256 is computed over the entire object including the header, so even two files with the same content but stored under different types would produce different hashes. The filename in the object store IS the hash — so no separate index of hashes is ever needed.

Phase 3 — The Index (Staging Area)

Phase 3 implements the staging area — the preparation zone for the next commit. The index is a plain text file at .pes/index with one entry per line: mode, hash, mtime, size, path. Three functions are implemented: index_load (parse the file), index_save (write atomically), and index_add (stage a file by hashing and recording it).

Screenshot 3A — pes init, pes add, pes status

Q: Implement the staging area. Show that pes add stages files correctly and pes status reports the right state.

```
karan@karanvm:~/PES2UG24CS903-pes-vcs$ make all
rm -rf .pes file1.txt file2.txt
./pes init
echo "hello world" > file1.txt
echo "goodbye" > file2.txt
./pes add file1.txt file2.txt
./pes status
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
    staged: file1.txt
    staged: file2.txt

Unstaged changes:
    (nothing to show)
```

After ./pes init creates the repository structure, ./pes add file1.txt file2.txt reads each file, writes it as a blob object, and records an entry in .pes/index with the blob hash and file metadata. ./pes status then loads the index and compares each entry's mtime and size against the current on-disk file. Since the files were just written, their metadata matches perfectly, so both appear under 'Staged changes' with no unstaged or untracked files. Index saves are atomic: written to .pes/index.tmp first, then renamed, so a crash never corrupts the index.

Screenshot 3B — Index File Contents

Q: Show the raw contents of .pes/index to demonstrate the text-based format.

```
karan@karanvm:~/PES2UG24CS903-pes-vcs$ cat .pes/index
100644 0bd69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d 1776667203 12 file1.txt
100644 b17b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92 1776667203 8 file2.txt
karan@karanvm:~/PES2UG24CS903-pes-vcs$
```

The index file contains two lines, one per staged file. Each line has five fields: 100644 (octal file mode for a regular non-executable file), the 64-character hex SHA-256 of the staged blob, the mtime in seconds since epoch (1776667203), the file size in bytes (12 for 'hello world\n', 8 for 'goodbye\n'), and the relative file path. Entries are sorted by path, making the file deterministic regardless of the order files were added. The mtime and size fields enable fast change detection in pes status without re-reading or re-hashing every file.

Phase 4 — Commits and History

Phase 4 implements commits — the objects that tie everything together. A commit stores the root tree hash (the full project snapshot), an optional parent hash (the previous commit), author name, Unix timestamp, and a commit message. The functions implemented are commit_create (build and store a commit) and cmd_commit (CLI parsing). head_update atomically swings the branch pointer after each commit.

Screenshot 4A — pes log (Commit History)

Q: Implement commit_create. Show pes log displaying the full commit history after making three commits.

```
commit.c: In function ‘head_update’:
commit.c:122:45: warning: ‘.tmp’ directive output may be truncated writing 4 bytes into a region of size between 1 and 512 [
  122 |     snprintf(tmp_path, sizeof(tmp_path), "%s.tmp", target_path);
      |                                              ^~~~
In file included from /usr/include/stdio.h:894,
                 from pes.h:8,
                 from commit.h:9,
                 from commit.c:1:
/usr/include/aarch64-linux-gnu/bits/stdio2.h:71:10: note: ‘__builtin___snprintf_chk’ output between 5 and 516 bytes into a d
   71 |   return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
      |          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
      |                                     __glibc_objsize (__s), __fmt,
      |                                     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   73 |                                     __va_arg_pack ());
      |                                     ~~~~~~~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Committed: 59eca80c6d2f... Initial commit
Committed: e450589e1209... Add world
Committed: b99748b1fd83... Add farewell
commit b99748b1fd8322b2132686bb776c42b62fb6dc1aaaa580b01b210324e14b2dc0
Author: Karan Varshney <PES2UG24CS903>
Date:   1776668397

    Add farewell

commit e450589e1209ee673f88a8dd84ae0740742952cfe2bc2eed954d454890f3dbcd
Author: Karan Varshney <PES2UG24CS903>
Date:   1776668397

    Add world

commit 59eca80c6d2fb6d82cfa609f13d14ed935a412c89a797da7d1aba33832336352
Author: Karan Varshney <PES2UG24CS903>
Date:   1776668397

    Initial commit
```

Three commits are shown newest-first: 'Add farewell', 'Add world', and 'Initial commit'. Each commit shows its full 64-character SHA-256 hash, Author: Karan Varshney , a Unix timestamp, and the message. commit_walk follows the parent chain: it reads HEAD to get the latest commit hash, reads that commit object, calls the callback (print), then reads the parent commit and repeats until reaching the root commit (no parent field). The linked list of commits on disk is exactly how Git stores history — no separate history file is needed.

Screenshot 4B — Object Store After 3 Commits

Q: Show the full .pes directory structure after three commits to demonstrate object store growth.

```
karan@karanvm:~/PES2UG24CS903-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/59/eca80c6d2fb6d82cfa609f13d14ed935a412c89a797da7d1aba33832336352
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/b9/9748b1fd8322b2132686bb776c42b62fb6dc1aaaa580b01b210324e14b2dc0
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e4/50589e1209ee673f88a8dd84ae0740742952cfe2bc2eed954d454890f3dbcd
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
karan@karanvm:~/PES2UG24CS903-pes-vcs$
```

After three commits the .pes directory contains: .pes/HEAD (symbolic ref), .pes/index (staging area), .pes/refs/heads/main (branch pointer), and 10 object files. The 10 objects include: 3 commit objects (one per commit), 3 root tree objects (one snapshot per commit), and 4 blob objects (hello.txt version 1, hello.txt version 2 after appending 'World', bye.txt, and one from Phase 1 testing). The unchanged Makefile and other source files are NOT stored as blobs because they were never staged — demonstrating that PES-VCS only stores what you explicitly add.

Screenshot 4C — Reference Chain: HEAD → main → commit hash

Q: Show how HEAD and refs/heads/main form the reference chain that tracks the current commit.

```
karan@karanvm:~/PES2UG24CS903-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
b99748b1fd8322b2132686bb776c42b62fb6dc1aaaa580b01b210324e14b2dc0
ref: refs/heads/main
karan@karanvm:~/PES2UG24CS903-pes-vcs$
```

.pes/refs/heads/main contains the raw SHA-256 of the latest commit (b99748b1fd83...). .pes/HEAD contains 'ref: refs/heads/main' — a symbolic reference that follows the main branch automatically. This two-level indirection is what makes branching cheap: creating a new branch is just creating a new file in .pes/refs/heads/. head_update writes the new commit hash atomically (temp file + rename) into the branch file, so the pointer swing is always either fully complete or not done at all — a crash between commits leaves the previous commit still intact.

Final — Integration Test

The integration test (make test-integration) runs the shell script test_sequence.sh which exercises all four phases end-to-end: repository initialization, staging files, making three commits, verifying history, checking the reference chain, and counting objects.

Screenshot Final — make test-integration

Q: Run the full end-to-end integration test to verify all phases work together correctly.

```
karan@karanvm:~/PES2UG24CS903-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
    staged: file.txt
    staged: hello.txt

Unstaged changes:
    (nothing to show)

Untracked files:
    (nothing to show)


--- First Commit ---
Committed: 9d71f9613e3c... Initial commit

Log after first commit:
commit 9d71f9613e3cebaa673803c5df474e2f44a54350a5a861e51f83842c2510b8c3
Author: Karan Varshney <PES2UG24CS903>
Date:   1776668627

    Initial commit


--- Second Commit ---
Committed: ad6c11ed147c... Update file.txt

--- Third Commit ---
Committed: 0778e3b5e669... Add farewell

--- Full History ---
commit 0778e3b5e6694c54532113d2aa2fbf05770a59014ea5e83d15909335df2cb3aa
```

```
--- Full History ---
commit 0778e3b5e6694c54532113d2aa2fbf05770a59014ea5e83d15909335df2cb3aa
Author: Karan Varshney <PES2UG24CS903>
Date:   1776668627

    Add farewell

commit ad6c11ed147cd9b68d27b5d65bf5c2f1482fc0668032071247d2442298167522
Author: Karan Varshney <PES2UG24CS903>
Date:   1776668627

    Update file.txt

commit 9d71f9613e3cebaa673803c5df474e2f44a54350a5a861e51f83842c2510b8c3
Author: Karan Varshney <PES2UG24CS903>
Date:   1776668627

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
0778e3b5e6694c54532113d2aa2fbf05770a59014ea5e83d15909335df2cb3aa

--- Object Store ---
Objects created:
10
.pes/objects/07/78e3b5e6694c54532113d2aa2fbf05770a59014ea5e83d15909335df2cb3aa
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/9d/71f9613e3cebaa673803c5df474e2f44a54350a5a861e51f83842c2510b8c3
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/ad/6c11ed147cd9b68d27b5d65bf5c2f1482fc0668032071247d2442298167522
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
karan@karanvm:~/PES2UG24CS903-pes-vcs$
```

The integration test passes all sections: Repository Initialization (PASS for .pes/objects, .pes/refs/heads, .pes/HEAD), Staging Files (two files staged and shown in status), First/Second/Third Commit (each produces the correct 'Committed: hash... message' output), Full History (all three commits shown in log with correct author and timestamps), Reference Chain (HEAD → refs/heads/main → latest commit hash verified), and Object Store (exactly 10 objects created). All integration tests completed successfully.

PART-B — Analysis Questions

Q5.1 — How would you implement pes checkout? What files change and what makes it complex?

A branch in PES-VCS is just a file in .pes/refs/heads/ containing a commit hash. Creating a branch is creating a file — an O(1) operation.

Files that change in .pes/: .pes/HEAD is rewritten to 'ref: refs/heads/<branch>'. If the branch is new, .pes/refs/heads/<branch> is created with the current commit hash.

Working directory update: The target commit's root tree is read from the object store. Every tree entry is walked recursively — blob entries restore file content from the object store, tree entries recurse into subdirectories. Files present in the current branch but absent from the target must be deleted.

What makes checkout complex: The main challenge is conflict detection. For every tracked file, its on-disk mtime and size must be compared against the index. If a file is locally modified AND differs between the two branches, checkout must refuse to overwrite it — otherwise the user's unsaved work is permanently lost. The operation must also be atomic: all file writes complete before HEAD is updated, so a crash mid-checkout does not leave the repository half-switched.

Q5.2 — How would you detect a dirty working directory using only the index and object store?

When switching branches, if a tracked file has local uncommitted changes AND differs between the two branches, checkout must refuse. Detection uses only the index and object store:

Step 1 — Fast metadata check: For every file in the index, stat() the on-disk file and compare its mtime and size to the mtime_sec and size fields in the index entry. If either differs, the file is dirty. This is the same fast-path used by pes status.

Step 2 — Hash verification (optional deeper check): If metadata matches, the file is almost certainly unchanged. For certainty, re-read the file, compute its SHA-256, and compare against the blob hash stored in the index entry.

Step 3 — Cross-branch diff: For each dirty file, compare its blob hash in the current branch's tree against the blob hash in the target branch's tree. If they differ, checkout must abort — overwriting would destroy local changes. If they are the same, the file content is identical across branches and checkout can safely proceed even if the file appears locally modified.

This approach avoids re-hashing every clean file by using mtime/size as a fast O(1) filter.

Q5.3 — What is detached HEAD? What happens if you commit in that state and how do you recover?

Detached HEAD means .pes/HEAD contains a raw commit hash directly (e.g., 'a1b2c3...') instead of a symbolic branch reference like 'ref: refs/heads/main'. This happens when checking out a specific commit hash rather than a branch name.

What happens when you commit in detached HEAD state: New commits are created and chained correctly — each commit's parent field points to the previous one, and .pes/HEAD is updated to the new hash. However, no branch file in .pes/refs/heads/ is ever updated. The commits exist in the object store but are floating — no named reference points to them.

The danger: Once you checkout a branch, HEAD moves away. The floating commits have no ref pointing to them and become unreachable. A garbage collector would delete them because the mark phase never visits them.

Recovery: The user needs the hash of the last floating commit. In real Git, git reflog records every HEAD movement. Once found, the user runs: git branch recovery-branch <hash> — creating .pes/refs/heads/recovery-branch containing that hash, making the entire commit chain reachable and safe from GC.

Q6.1 — Describe a garbage collection algorithm for PES-VCS. What data structure and how does it scale?

Unreachable objects accumulate over time — replaced file blobs, superseded tree snapshots, or floating commits from detached HEAD work. GC finds and deletes them.

Mark Phase: Read every file in .pes/refs/heads/ and follow .pes/HEAD. For each ref, read the commit object. Record its tree hash and walk the tree recursively — add every blob hash and subtree hash to the reachable set. Follow the commit's parent pointer and repeat until the root commit (no parent). Repeat for every branch.

Sweep Phase: Walk every file under .pes/objects/XX/. For each file, reconstruct its hash from the shard directory (XX) + filename. If NOT in the reachable set, delete the file.

Data structure: A hash set using open-addressing with 32-byte SHA-256 keys provides O(1) average insertion and lookup. A sorted array with binary search gives O(log n) but better cache locality for large sets. Both are acceptable.

Scale estimate for 100,000 commits and 50 branches: With an average of 3 objects per commit (1 commit + 1 tree + ~1 blob), roughly 300,000 objects must be visited during marking. With shared history across branches this is much less. The sweep must scan all object files which may number in the millions for a mature repository.

Q6.2 — Why is concurrent GC and commit dangerous? Describe the race condition and how Git avoids it.

Running GC concurrently with an active commit creates a classic TOCTOU (Time-Of-Check to Time-Of-Use) race condition that can silently corrupt the repository.

The race condition: (1) A concurrent commit calls object_write() and successfully writes a new blob to .pes/objects/. The commit has not yet called head_update() — so no ref points to this blob. (2) GC runs its mark phase, reads all refs, walks all reachable objects. The new blob is not reachable yet because head_update() has not run. (3) GC's sweep phase deletes the blob as unreachable. (4) The commit calls head_update(). HEAD now points to a commit whose tree references a blob that no longer exists. The repository is permanently corrupted — any checkout or log will fail.

How real Git avoids this: Git uses a grace period — any object newer than a configurable threshold (default 14 days, gc.pruneExpire) is never deleted by GC regardless of reachability. This gives all in-progress operations ample time to complete and update their refs before GC can touch new objects.

Git also uses .lock files on ref files during commit operations, so GC can detect an active write and defer. Combined with atomic object writes (temp file + rename, identical to our object_write implementation), these three mechanisms — grace period, lock files, and atomic writes — make concurrent GC safe in practice.